## Nama : Oktova Yoga Praditia Samosir

## Nim : 42324001

## Prodi : D3 - Teknologi Informasi

In [ ]:
```
!nvidia-smi
```

!nvidia-smi menampilkan informasi perangkat gpu yang tersedia pada runtime, seperti nama gpu, penggunaan memori, suhu, dan versi driver/cuda. perangkat gpu yang terdeteksi di colab. jika tidak ada gpu, tensorflow tetap dapat berjalan menggunakan cpu, tetapi training gambar biasanya lebih lambat.

In [ ]:
```
!pip install roboflow

from roboflow import Roboflow
rf = Roboflow(api_key="LsuywI3tHluMjbVVnZn9")
project = rf.workspace("roboflow-universe-projects").project("banana-ripeness-class
version = project.version(4)
dataset = version.download("folder")

!mv /content/Banana-Ripeness-Classification-4/valid /content/Banana-Ripeness-Classi
```

cell ini memasang library roboflow, lalu membuat koneksi ke roboflow menggunakan api key dan mengambil project banana-ripeness-classification versi 4. download('folder') mengunduh dataset dalam struktur folder yang dapat dibaca oleh generator gambar.

dataset berisi gambar pisang untuk empat kelas kematangan: overripe, ripe, rotten, dan unripe. perintah mv mengganti nama folder valid menjadi val agar cocok dengan alamat folder yang dipakai pada cell berikutnya.

In [ ]:
```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Direktori dataset
train_dir = dataset.location + "/train"
valid_dir = dataset.location + "/val"

# Augmentasi data dan preprocessing
train_datagen = ImageDataGenerator(
    rescale=1./255,          # Normalisasi gambar
    rotation_range=20,       # Rotasi acak
    width_shift_range=0.2,   # Pergeseran horizontal
    height_shift_range=0.2, # Pergeseran vertikal
    shear_range=0.2,         # Perubahan sudut
    zoom_range=0.2,          # Zoom acak
    horizontal_flip=True     # Membalik gambar secara horizontal
)
```

```
# Untuk set validasi, kita hanya melakukan rescale
valid_datagen = ImageDataGenerator(rescale=1./255)

# Membuat data generator
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(224, 224),   # Ubah ukuran gambar
    batch_size=32,
    class_mode='categorical'  # Klasifikasi multi-kelas
)

valid_generator = valid_datagen.flow_from_directory(
    valid_dir,
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical'
)
```

cell ini menyiapkan gambar agar formatnya sesuai dengan input cnn. imagedatagenerator membuat generator yang membaca gambar per batch, sehingga seluruh gambar gk harus dimuat ke memori bersamaan.

pada data training:

- rescale=1./255 mengubah rentang nilai piksel dari 0–255 menjadi 0–1.
- rotation_range, width_shift_range, dan height_shift_range memberi variasi rotasi serta pergeseran gambar.
- shear_range dan zoom_range memberi variasi kemiringan dan pembesaran.
- horizontal_flip=true membalik gambar secara horizontal. augmentasi membantu model melihat variasi tampilan gambar dan mengurangi kecenderungan menghafal contoh training. untuk validasi hanya dilakukan rescale agar gambar penilaian tidak diubah secara acak.

flow_from_directory() membaca gambar berdasarkan nama subfolder. target_size=(224,224) menyamakan ukuran gambar, batch_size=32 memproses 32 gambar per batch, dan class_mode='categorical' menyiapkan label untuk klasifikasi empat kelas. hasil generator biasanya menampilkan jumlah gambar dan jumlah kelas yang ditemukan.

In [ ]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

# Membangun model CNN
model = Sequential()

# Lapisan Convolutional pertama
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(224, 224, 3)))
model.add(MaxPooling2D(pool_size=(2, 2)))

# Lapisan Convolutional kedua
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
```

```
# Lapisan Convolutional ketiga
model.add(Conv2D(128, (3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

# Lapisan Flatten
model.add(Flatten())

# Fully Connected Layer
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))  # Dropout untuk mencegah overfitting

# Lapisan Output
model.add(Dense(4, activation='softmax'))  # 4 kelas untuk kematangan pisang

# Kompilasi model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy

# Menampilkan ringkasan model
model.summary()
```

## penjelasan membangun arsitektur cnn

model dibangun berurutan menggunakan sequential, cnn mengekstraksi ciri gambar melalui convolution, mengurangi ukuran representasi dengan pooling, lalu menggabungkan ciri untuk klasifikasi.

- conv2d(32, (3,3)) menggunakan 32 filter/kernel berukuran 3×3. setiap kernel bergerak di atas area gambar, mengalikan nilai piksel dengan bobot kernel, lalu menjumlahkannya untuk membentuk feature map. filter belajar mendeteksi pola seperti tepi, tekstur, dan bagian bentuk.
- maxpooling2d((2,2)) mengambil nilai maksimum dari setiap area 2×2. pooling mengecilkan dimensi spasial feature map sehingga komputasi lebih ringan dan ciri dominan dipertahankan.
- blok berikutnya memakai 64 lalu 128 filter agar jaringan dapat mempelajari representasi yang semakin banyak dan kompleks.
- aktivasi relu mengubah nilai negatif menjadi nol dan mempertahankan nilai positif, sehingga jaringan dapat mempelajari hubungan non-linear.
- flatten() meratakan feature map menjadi vektor.
- dense(128) menggabungkan fitur yang telah diekstraksi untuk menentukan kelas.
- dropout(0.5) menonaktifkan secara acak 50% unit saat training untuk membantu mengurangi overfitting.
- dense(4, activation='softmax') menghasilkan empat skor probabilitas kelas. softmax mengubah skor menjadi distribusi probabilitas dengan total 1. compile() menetapkan optimizer adam, loss categorical crossentropy karena label berupa empat kelas, dan accuracy sebagai metrik. model.summary() menampilkan bentuk output dan jumlah parameter tiap layer.

In [ ]:
```
# Melatih model
history = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // train_generator.batch_size,
    validation_data=valid_generator,
    validation_steps=valid_generator.samples // valid_generator.batch_size,
    epochs=9  # Jumlah epoch
)
```

model.fit() melatih cnn menggunakan train_generator. pada setiap batch, gambar melewati convolution untuk menghasilkan feature map, pooling untuk mengurangi dimensi, lalu flatten dan dense untuk menghasilkan prediksi kelas.

prediksi dibandingkan dengan label sebenarnya menggunakan categorical crossentropy. selanjutnya backpropagation menghitung gradien kesalahan dan optimizer adam memperbarui bobot filter serta layer dense. proses ini diulang selama beberapa epoch.

steps_per_epoch menentukan jumlah batch training dalam satu epoch, sedangkan validation_data dan validation_steps digunakan untuk menghitung performa pada data validasi tanpa memperbarui bobot. epochs=9 berarti jumlah putaran training yang diminta maksimal sembilan kali.

output menampilkan accuracy dan loss training serta val_accuracy dan val_loss. accuracy yang meningkat dan loss yang menurun biasanya menunjukkan pembelajaran membaik. pada output notebook terdapat peringatan your input ran out of data; ini berarti generator kehabisan batch sebelum langkah yang diminta selesai, sehingga training yang tersimpan dapat tidak berjalan sesuai rencana. hasil training perlu diperiksa lagi dan sebaiknya langkah generator disesuaikan atau gunakan model.fit() tanpa perhitungan langkah manual jika generator sudah mengetahui panjang datanya.

In [ ]:
```
# Model evaluation using precision, recall, confution matrix and f1 score, display

from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import numpy as np

# Evaluasi model pada keseluraun class data validasi
val_loss, val_accuracy = model.evaluate(valid_generator)
print(f"Validation Loss: {val_loss}")
print(f"Validation Accuracy: {val_accuracy}")

# Evaluasi model pada setiap class data validasi
y_pred = model.predict(valid_generator)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = valid_generator.classes

# Menghitung metrik evaluasi
report = classification_report(y_true, y_pred_classes)
print(report)
```

```
# Membuat confusion matrix
cm = confusion_matrix(y_true, y_pred_classes)

# Menampilkan confusion matrix
plt.figure(figsize=(8, 6))
plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
plt.title('Confusion Matrix')
plt.colorbar()
tick_marks = np.arange(len(valid_generator.class_indices))
plt.xticks(tick_marks, valid_generator.class_indices, rotation=45)
plt.yticks(tick_marks, valid_generator.class_indices)

# Menambahkan label pada setiap sel
thresh = cm.max() / 2.
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, format(cm[i, j], 'd'),
                 horizontalalignment="center",
                 color="white" if cm[i, j] > thresh else "black")

plt.tight_layout()
plt.ylabel('True label')
plt.xlabel('Predicted label')
plt.show()
```

model.evaluate(valid_generator) menghitung loss dan accuracy pada data validasi. output ini menmplkan validation accuracy sekitar 0.8851 atau 88.51% dan validation loss sekitar 0.4507.

model.predict() menghasilkan probabilitas untuk setiap kelas. argmax(axis=1) memilih indeks kelas dengan probabilitas tertinggi. valid_generator.classes berisi label sebenarnya. classification_report() merangkum precision, recall, f1-score, dan support untuk tiap kelas:

- precision: dari semua prediksi pada suatu kelas, berapa bagian yang benar.
- recall: dari semua data aktual pada suatu kelas, berapa bagian yang berhasil ditemukan.
- f1-score: gabungan precision dan recall menggunakan rata-rata harmonik.
- support: jumlah data aktual pada kelas tersebut. confusion_matrix() membandingkan label aktual dengan label prediksi. baris menunjukkan kelas sebenarnya dan kolom menunjukkan kelas prediksi; nilai diagonal adalah prediksi benar, sedangkan nilai di luar diagonal adalah kesalahan antar kelas. kode berikutnya menampilkan matriks tersebut dan menuliskan jumlah pada setiap sel.

In [ ]:
```
import matplotlib.pyplot as plt

# Visualisasi akurasi
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.legend()
plt.show()

# Visualisasi loss
plt.plot(history.history['loss'], label='Training Loss')
```

```
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.legend()
plt.show()
```

objek history dari proses training menyimpan nilai metrik pada setiap epoch. grafik pertama membandingkan training accuracy dengan validation accuracy, sedangkan grafik kedua membandingkan training loss dengan validation loss.

accuracy menunjukkan proporsi prediksi kelas yang benar. loss menunjukkan besar kesalahan prediksi berdasarkan categorical crossentropy. kurva training dan validation yang sama-sama membaik menunjukkan model belajar dan mampu bekerja pada data validasi. jika training membaik tetapi validation memburuk, hal itu dapat menjadi tanda overfitting.

grafik harus dibaca bersama output training sebelumnya. karena training menampilkan peringatan kehabisan data, kurva yang terbentuk perlu dibaca atau diartikn dengan hati-hati dan training sebaiknya dijalankan ulang dengan langkah generator yang benar.

In [ ]:
```
# inference using model, and add text to displayed predict image

from tensorflow.keras.preprocessing import image
import matplotlib.pyplot as plt
import numpy as np

def predict_image(image_path):
  img = image.load_img(image_path, target_size=(224, 224))
  img_array = image.img_to_array(img)
  img_array = np.expand_dims(img_array, axis=0)
  img_array /= 255.

  prediction = model.predict(img_array)
  predicted_class = np.argmax(prediction)

  # Get class labels from the generator
  class_labels = list(train_generator.class_indices.keys())

  # Display the image with prediction
  plt.imshow(img)
  plt.title(f"Predicted label: {class_labels[predicted_class]}")
  plt.axis('off')
  plt.show()

# Example usage
image_path = "/content/Banana-Ripeness-Classification-4/test/ripe/musa-acuminata-ba
predict_image(image_path)
```

fungsi predict_image() digunakan untuk mencoba model pada satu gambar. load_img() mengubah ukuran gambar menjadi 224×224, img_to_array() mengubah gambar menjadi array numerik, dan expand_dims() menambahkan dimensi batch karena model menerima input dalam bentuk kumpulan gambar.

pembagian dengan 255 menyamakan preprocessing gambar uji dengan data training. model predict menghasilkan probabilitas untuk empat kelas, lalu argmax() mengambil indeks dengan nilai tertinggi. indeks tersebut dicocokkan dengan nama kelas dari train generator.class indices.

hasil yang ditampilkan adalah gambar uji dengan label prediksi pada judul. ini merupakan inference, yaitu penggunaan model yang sudah dilatih untuk memprediksi data baru, bukan proses memperbarui bobot.

# YOLOv8 Transfer Learning

## Install YOLOv8

In [ ]:
```
# Pip install method (recommended)

!pip install ultralytics

from IPython import display
display.clear_output()

import ultralytics
ultralytics.checks()
```

pip install ultralytics memasang paket ultralytics yang menyediakan implementasi yolo. display.clear_output() membersihkan tampilan output instalasi agar notebook lebih rapi. ultralytics.checks() memeriksa apakah semuanya sudah siap, termasuk versi paket dan ketersediaan hardwarenya

output menunjukkan versi yolov8 yang digunakan dan gpu tesla t4 terdeteksi. ini menandakan lingkungan siap untuk menjalankan perintah training, validation, dan prediction melalui command line interface (cli).

## Inference with Pre-trained COCO Model

In [ ]:
```
!yolo task=classify mode=predict model=yolov8x-cls.pt conf=0.25 source='https://med
```

perintah yang dijelaskan pada modul. task=classify memilih tugas klasifikasi gambar, mode=predict menjalankan prediksi, dan model=yolov8x-cls.pt menggunakan bobot pretrained untuk klasifikasi yang sudah dilatih sebelumnya.

conf=0.25 menetapkan ambang confidence 0.25, sedangkan source menunjukkan gambar anjing yang akan diprediksi. model pretrained ini digunakan sebagai contoh inference sebelum model disesuaikan dengan dataset pisang.

output memperlihatkan model dimuat dan gambar diproses. hasil klasifikasi menampilkan kelas yang diprediksi beserta skor confidence. ini menunjukkan model pretrained dapat langsung digunakan, tetapi belum berarti model tersebut sudah mengenali empat kategori kematangan pisang.

In [ ]:
```
from IPython.display import Image
Image(filename='runs/classify/predict/dog.jpeg', height=600)
```

image dari ipython.display digunakan untuk menampilkan file gambar hasil inference yang disimpan pada runs/classify/predict/argumen height=600 mengatur tinggi tampilan gambar yaitu gambar dog.jpeg.

hanya memperlihatkan gambar yang sudah diproses oleh perintah yolo sebelumnya agar hasil visual dapat diperiksa.

# Custom Training

In [ ]:
```
!yolo task=classify mode=train model=yolov8x-cls.pt data={dataset.location} epochs=
```

perintah ini melatih model klasifikasi yolov8 menggunakan dataset pisang. model=yolov8x-cls.pt memulai dari bobot pretrained, sehingga model tidak belajar seluruh fitur dari nol. pendekatan ini disebut transfer learning: pengetahuan dari model yang telah dilatih pada dataset besar dimanfaatkan untuk tugas baru.

data={dataset.location} menunjukkan lokasi dataset dengan folder train, val, dan test. epochs=5 menentukan lima epoch, imgsz=224 mengubah ukuran input menjadi 224 piksel, batch=32 memproses 32 gambar per batch, dan freeze=5 membekukan lima layer awal agar bobot awalnya tidak ikut diperbarui. bagian yang tidak dibekukan dapat menyesuaikan diri dengan empat kelas pisang.

In [ ]:
```
# prompt: show image from runs/classify/train/result.png

from IPython.display import Image
Image('/content/runs/classify/train/results.png', width=750)
```

image() menampilkan file results.png yang dibuat otomatis oleh ultralytics selama training. file ini berisi grafik perkembangan metrik seperti loss dan accuracy selama epoch.

grafik inii membantu melihat apakah kesalahan model menurun dan performa klasifikasi meningkat. grafik training perlu dibandingkan dengan grafik validasi untuk mengamati kemungkinan overfitting. cell ini hanya menampilkan ringkasan training yang sudah tersimpan, bukan melatih model kembali.

# Validate Custom Model

In [ ]:
```
!yolo task=classify mode=val model=/content/runs/classify/train/weights/best.pt dat
```

perintah mode=val menjalankan evaluasi model yang sudah dilatih pada bagian validasi dataset. model /best.pt memilih bobot terbaik yang disimpan selama training, sedangkan data={dataset.location} menentukan dataset yang digunakan.

In [ ]:
```
Image('/content/runs/classify/val/confusion_matrix_normalized.png', width=750)
```

cell ini menampilkan gambar yang dihasilkan saat validation. confusion matrix memperlihatkan hubungan kelas aktual dan kelas yang diprediksi model. nilai diagonal menunjukkan prediksi benar, sedangkan nilai di luar diagonal menunjukkan kelas yang tertukar.

# Inference menggunakan Custom Model

In [ ]:
```
!yolo task=classify mode=predict model=/content/runs/classify/train/weights/best.pt
```

menjalankan mode=predict dengan custom model best.pt pada folder gambar test kelas overripe. confidence threshold ditetapkan 0.25. model memproses gambar satu per satu dan mengeluarkan skor untuk setiap kelas kematangan.

In [ ]:
```
!yolo task=classify mode=predict model=/content/runs/classify/train/weights/best.pt
```

menggunakan custom model yang sama, tetapi sumber gambar diarahkan ke folder test/ripe. tujuan pengujian adalah melihat apakah model dapat mengenali gambar pisang matang (ripe) dari data uji.

output menampilkan kelas yang diprediksi dan confidence untuk setiap gambar.

In [ ]:
```
!yolo task=classify mode=predict model=/content/runs/classify/train/weights/best.pt
```

menguji model pada gambar di folder test/rotten. semua parameter tetap sama seperti cell sebelumnya; yang berubah hanya folder sumber gambar.

output memperlihatkan prediksi kelas dan confidence. hasil yang sesuai dengan rotten menunjukkan model berhasil mengenali contoh tersebut

In [ ]:
```
!yolo task=classify mode=predict model=/content/runs/classify/train/weights/best.pt
```

cell ini menjalankan model pada gambar di folder test/unripe. tujuannya memeriksa kemampuan model mengenali pisang yang belum matang.

output berisi kelas prediksi dan confidence untuk setiap gambar.

In [ ]:
```
import glob
from IPython.display import Image, display
```

```
for image_path in glob.glob(f'/content/runs/classify/predict2/*.jpg')[:3]:
      display(Image(filename=image_path, width=600))
```

glob.glob() mencari file gambar pada folder hasil prediksi runs/classify/predict2/. pemotongan [:3] membatasi tampilan maksimal tiga gambar, sedangkan display(image()) menampilkan gambar satu per satu.

In [ ]:
```
import glob
from IPython.display import Image, display

for image_path in glob.glob(f'/content/runs/classify/predict3/*.jpg')[:3]:
      display(Image(filename=image_path, width=600))
```

kode ini sama seperti cell sebelumnya, tetapi mengambil gambar dari folder runs/classify/predict3/

In [ ]:
```
import glob
from IPython.display import Image, display

for image_path in glob.glob(f'/content/runs/classify/predict4/*.jpg')[:3]:
      display(Image(filename=image_path, width=600))
```

cell ini mengambil maksimal tiga file gambar dari folder runs/classify/predict4/ lalu menampilkannya

In [ ]:
```
import glob
from IPython.display import Image, display

for image_path in glob.glob(f'/content/runs/classify/predict5/*.jpg')[:3]:
      display(Image(filename=image_path, width=600))
```

mengambil maksimal tiga gambar dari folder runs/classify/predict5/ dan menampilkannya. folder ini berisi hasil inference untuk sumber gambar kelas unripe.